Machine Learning for Molecular Engineering
3/7/10/20.01 (U) 3/7/10/20.C51 (G)
Spring 2025

Problem Set #3

Date: April 14, 2025
Due: April 23, 2025 @ 3:00 pm ET

Instructions

- This problem set has two modeling tasks with several sub-questions. Some are marked grad version, which are required for graduate students (X.C51) but optional for others. Points for all students are in blue, while grad-only points are in orange. There is one problem that is undergrad only in purple. The total points are 75 for undergraduates and 100 for graduates.
- To get started, open your Google Colab or Jupyter notebook starting from the problem set template file pset3.ipynb (direct Colab link). You will need to use the data files here. Make your own copy of this template to save changes, by selecting "Save a copy in Drive".
- **Important:** This problem set requires a GPU. Before you start in Google Colab, find `Notebook Settings` under the `Edit` menu. In the `Hardware accelerator` drop-down, select a T4 GPU as your hardware accelerator. Changing the runtime resets the notebook, so make this change before starting!
- Collaboration is encouraged and AI tools are permitted, but submitting work that is not your own is plagiarism. Any collaboration or assistance from an LLM (including utilities present within Google Colab) should be described at the end of your submission.
- Upon completion, submit your IPython Notebook `pset3.ipynb` to the non-bio assignment on Gradescope. Ensure your submission includes *all* necessary code to be run without error, with all plots and outputs. Comments are encouraged; put answers to conceptual questions in Markdown/Text cells.

Background

You will learn how to develop graph neural networks to predict two molecular properties (separately):

1. the **aqueous solubility** of molecules. Aqueous solubility, the ability of a substance to dissolve in a water, is a key property in drug development as it directly influences the bioavailability of a drug. Poor solubility can lead to formulation challenges, impacting manufacturing, stability, and may necessitate specialized approaches to enhance solubility and optimize drug delivery.
2. the $\mathbf{IC}_{50}$ of molecules against a target protein's function. IC50, or the half-maximal inhibitory concentration, quantifies the potency of a molecule by indicating the concentration required to inhibit 50% of a target protein's biological function. For this PSET, you will look specifically at inhibitors screened against human β-secretase 1(BACE-1).

The main objectives of this problem set are to familiarize you with the internal architectures of (2D) Graph Neural Networks, including data processing and message-passing strategies; and to analyze the limitations of these GNN approaches for solving different property prediction tasks (namely, what happens when you try to use a 2D GNN and 2D representations to predict a property that requires knowledge of 3D information!).

Part 1: Predicting Solubility with a 2D GNN

In this part, you will develop a graph neural network (GNN) to predict the solubility of a molecule. We will use subset of the AqSolDB dataset [1] containing experimentally determined aqueous solubilities (unit: log mol/l). Solubility is crucial for drug development because it influences a drug's absorption and distribution within the body, affecting its bioavailability and therapeutic efficacy. Poorly soluble drugs may not reach therapeutic levels in the bloodstream, limiting their effectiveness. One distinguishing factor to keep in mind between this property and the one in Problem 2 is that these solubilities are in water, which is a *achiral* solvent; this means it does not induce any chiral interactions with dissolved molecules.

Background on Molecular Graphs and SMILES

Molecular Graphs

A graph $\mathcal{G}$ is a mathematical object that models connections (edges) between objects (nodes). Graph-structured data can be found in communication networks, ecological systems, and social networks. It is natural to consider using graphical representations for molecules because a molecule can be thought of as a set of atoms connected with chemical bonds.[1]

A graph $\mathcal{G}$ consists of a set of nodes $\mathcal{V}$ of size $n = |\mathcal{V}|$ and a set of edges $\mathcal{E}$ that represents the connections between pairs of nodes. The edges can be represented by a binary adjacency matrix $\mathbf{A} \in \{0,1\}^{n \times n}$ (Figure 1). For molecules, $\mathbf{A}$ is typically sparse since atoms are limited in terms of the number of connections they can make. Node $v_i \in \mathcal{V}$ and node $v_j \in \mathcal{V}$ are connected if $A_{ij} = 1$. $\mathbf{A}$ can be more compactly represented by an array of index pairs (i, j).

Nodes and edges of a graph $\mathcal{G}$ can be understood as data structure that stores chemical information for molecules just like how pixels in a image stores the information of colors. Based on the detailed chemistry of each molecule, the atoms (nodes) can have information about atomic numbers, formal charge, number of neighbors, etc., which can be stored in a vector of *node features*, $\mathbf{x}_i$. Edges can store information about bond orders, aromaticity, ring membership, etc., in vectors of *edge features*, $\mathbf{e}_{ij}$. *Edge features* and *node features* for a molecular graph can be constructed using various cheminformatics packages, and can capture structural information, properties computed through heuristics, or properties computed through quantum chemistry calculations.

SMILES: a string representation for molecules

The simplified molecular-input line-entry system (SMILES) is a text based notation describing the structure of molecules using short ASCII strings. In terms of a graph-based computational procedure, SMILES strings are generated by printing the symbol nodes encountered in a breadth-first traversal of the graphs, typically excluding hydrogen atoms. Any cycles are broken so that

[1] This is a bit of a simplification, particularly for molecules that exhibit stereochemistry, but good enough for most applications.

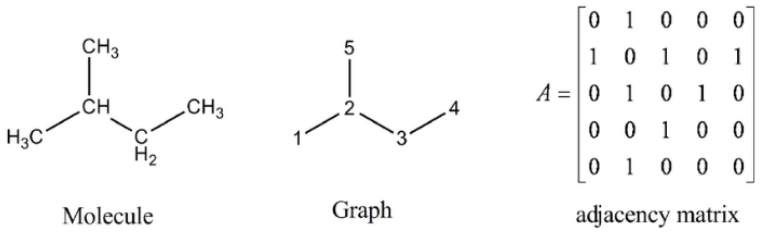

Figure 1: The chemical graph and adjacency matrix of isopentane [2]

the graph becomes an acyclic (tree-structured) graph. Numbers indicate connections between non-adjacent characters in the SMILES string; parentheses are used to indicate points of branching on the tree. Every molecule can be represented by multiple SMILES strings. For example, CCO, OCC, C(O)C, and [CH3][CH2][OH] all specify the structure of an ethanol molecule. There exist algorithms that can reproducibly generate a *canonical* SMILES string for a molecule. However, an arbitrary string of characters found in SMILES does not generate a SMILES string or a molecule; there is a fairly complex grammar that must be carefully respected.

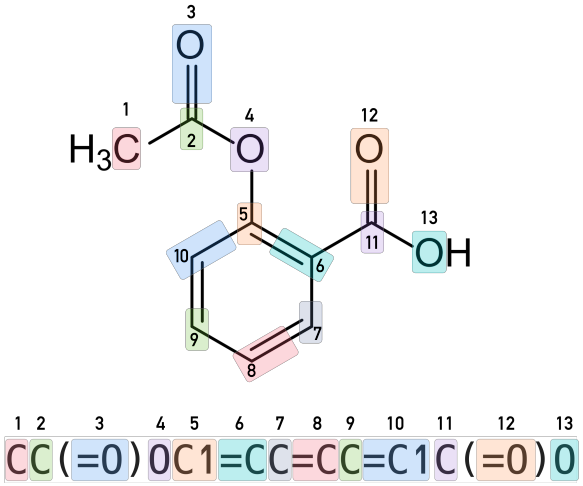

Figure 2: A molecular graph can be represented by SMILES string (image source)

Part 1.1 (5 points) Install and try out RDKit

RDKit is a open-source cheminformatics package that can process and manipulate molecular structures to work with molecules in their natural graph format. Follow the example code to create a `rdkit.Chem.rdchem.Mol` object and visualize it as a 2D line drawing.

Task: Choose 4 of your favorite molecules (or any molecules) and visualize their molecular graphs arranged in an image grid with `rdkit.Chem.Draw.MolsToGridImage`.

Part 1.2 (5 points) Collect atom and bond information for a sample molecule.

Figure 3 shows the molecular graph data structure for a GNN model. The GNN requires a feature set on the nodes and a list of edges as inputs. In the case of molecules, some possible feature sets might include atomic number, formal charge, aromaticity, and hybridization, though it is possible

(as you saw with the physical descriptors in PSET1) for feature sets to be quite extensive. A Graph Neural Network uses these node features and the edges to perform convolution operations and generate node-wise vector embeddings. A final readout layer takes the node-wise embedding to parameterize a pooled scalar (or vector) outputs. In many cases, an edge feature set is also included to encode chemical bond information like bond orders and aromaticity; for simplicity, the model you are going to work with only deals with node features.

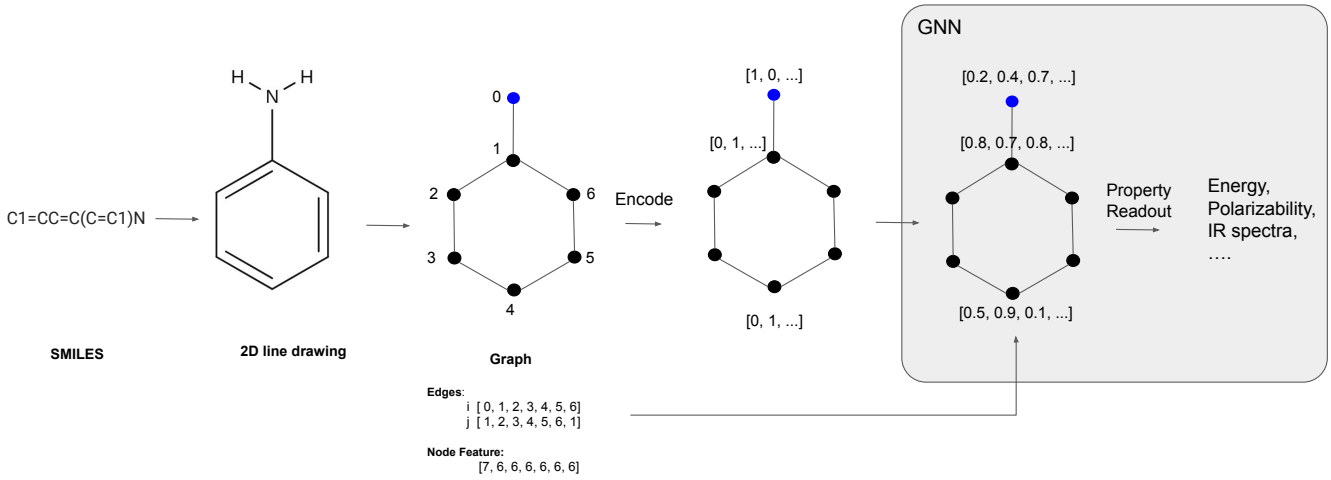

Figure 3: Data structure and model inputs for Graph Neural Networks

Before we can construct the GNN, we need to parse the molecules in our dataset with RDKit, which offers a comprehensive set of utilities for computing atomic features.

Task 1: Write a `smiles2features` function that takes in a SMILES and collects the above features per-atom in each molecule: atomic number, hybridization, aromaticity, attached hydrogen count, degree, and formal charge (in this order). For one molecule, your code should output a 2D np.array of atom features of shape (n_atoms, n_features) and a 2D np.array of edges of shape (2, n_edges * 2). These are the dimension matchings needed for consistency with other problems in the pset, so make sure these match up!

Part 1.3 (5 points) Process all samples in the dataset with smiles2features

Now that we have code to process each SMILES into node and edge arrays, we can collect this information for all of the graphs in the solubility dataset. We provide code to load the dataset as a `pandas.DataFrame` and shuffle its rows using `sklearn.utils.shuffle`.

Task 1: Loop over all the molecular SMILES and store the atomic numbers (`AtomicFeatures`), edge array (`Edge`), and the number of atoms ((`Natom`)) for each molecule and store them into three separate lists `Atomic_features_list`, `Edge_list`, and `Natom_list`. You also need to retrieve the solubility values (as a `torch.FloatTensor`) which are under the column name '*Solubility*' in your dataframe; this is the target property to predict. Wrap the solubility `y` values in 1-D array and store them all in a `y_list`. The data type required for `AtomicFeatures`, `Edge`, `Natom`, `y`, are `torch.LongTensor`, `torch.LongTensor` , `int`, and `torch.FloatTensor` (wrapped around a 1-D array) respectively.

Task 2: Look at a few samples after processing and answer the following 2 questions (a sentence for each is sufficient): 1) Is there any redundancy amongst the features we have provided? Phrased another way, is it possible to compute one of the atomic features from a combination of the others? (The answer may be no; ultimately, we're looking to see that you have made some interpretation of the features). 2) Does the `Natom_list` object keep counts that include the # of hydrogens? Why/why not? 3) What information about bonds have we retained, in `Edge_list`? Do we have any information about bond order? Might there be any atomic features that might give us some clues about bond order?

Part 1.4 (5 points) Construct molecular graph Datasets and DataLoaders

Task 1: Now, split your data into 70% training, 10% validation, and 20% testing data. You need to do this for `AtomicFeatures`, `Edge`, `Natom`, and `y`. Then, convert each of these sets into a `GraphDataset` object, which we have implemented for you.

With your `Dataset` object defined, you can feed them into a `DataLoader` (parts of which we have done for you). Batching a set of graphs is complicated because each graph has a different number of node and edges: you cannot simply batch them by stacking arrays of uneven lengths together. There are additional book-keeping procedures one needs to do: re-index node index in `Edge_batch` and record the sizes of graphs for the batch in `Natom_batch`. Figure 4 describes the batching operation.

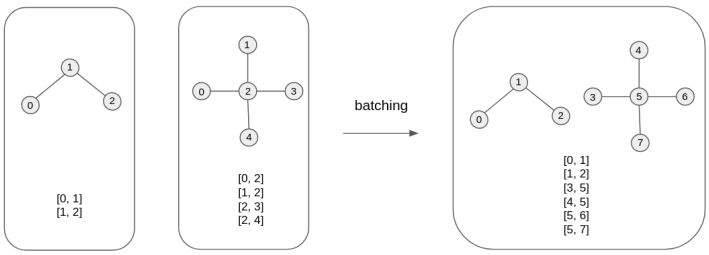

Figure 4: Batching operations for graphs

Because we cannot rely on standard stacking operations for making data batches, PyTorch allows us to define our own customized *collating* function, which takes a `list` of graph data tuple `(AtomicFeatures, Edge, Natom, y)` obtained from `GraphDataset.__getitem__()` method and lets us define how to combine them into one graph-structured data batch in a tuple `(AtomicFeatures_batch, Edge_batch, Natom_batch, y_batch)`. The function takes a list of tuples that contain the data, and return the single joined graph with node order re-indexed. We provide such a collation function, `collate_graphs()` which is passed along with `Datasets` to the`DataLoader` objects that you will construct. **Task 2:** Construct the DataLoaders with the Datasets you have made and `collate_graphs()`, and make sure you understand how the collation is performed. Set the batch size to 512; shuffle to True only for the train loader; and set `num_workers` to 2.

Part 1.5 Complete the definition of a GNN

For this problem, we recommend everyone read through this preamble on the layer construction in a GNN first, then complete only the subsection (Part 1.5.x) that pertains to their enrollment, which specifies what parts of the GNN they need to complete.

Embeddings

To build a Graph Neural Network (GNN), we first need an embedding layer for each of our features, followed by multiple message-passing and node-update steps. Since our goal is to predict a single scalar value for each molecule, we will then aggregate the node-level embeddings into a single representation using a dedicated aggregation layer.

Let j describe the feature index, and i describe the node index. Because each of our features do not share the same value range (e.g. a "0" for aromaticity does not mean the same thing as "0" for formal charge), we will need to embed each feature separately. Then, these featurized embeddings can be concatenated together to produce one node-level embedding that we will use for convolutions. To start, for each of our features $z^{(j)}$, we create an embedding layer:

$$h_i^{(j),t=0} = \mathbf{W}\texttt{onehot}(z_i^{(j)}) \tag{1}$$

For each feature j for a node z_i, let the size of the desired new feature dimension be $M^{(j)}$ and $N_{types}^{(j)}$ be the number of types that this feature can take on. For one node j at one node i, Equation 1 parameterizes the one-hot-encoded feature $\texttt{onehot}(z_i^{(j)}) \in \mathrm{R}^{\mathrm{N}_{\mathrm{types}}^{(\mathrm{j})}}$ to an initial embedding $h_i^{(j),t=0} \in \mathrm{R}^{\mathrm{M}^{(\mathrm{j})}}$. $\mathbf{W}$ is a $M^{(j)} \times N_{types}^{(j)}$ matrix. The embedding parametrization procedure can be done with the `torch.nn.Embeddings` module, which can take in a vector of (n_{nodes}) and produce a matrix of $(n_{(nodes)}, M^{(j)})$. Note from this definition that we will need to make a separate `nn.Embeddings` layer for each feature. A final note on `nn.Embedding`: it expects the minimum value in $N_{types}^{(j)}$ to be 0, and negative values are not supported. This will only affect the formal charge features, to which you can add 8 in the *forward* call to make everything minimally 0 (for the purposes of this problem, we can assume the lowest formal charge is -7).

As a design choice, because the $N_{types}^{(j)}$ might vary in range across different features, we might choose to have different $M^{(j)}$ depending on the level of expressivity. Upon constructing the feature embeddings, we just simply need to concatenate the feature matrices made with `torch.cat` so that we get a feature matrix of shape $H = (n_{nodes}, \sum_j (M^{(j)})$. Finally, we might pass a concatenated feature matrix into an initial linear layer (with `nn.Linear`) to learn between features and produces an initial feature embedding used for convolutions: $h^{t=0} = W_1 H$, where W_1 is a $n_{embed} \times \sum_j M^{(j)}$ matrix.

Here are the embedding layer sizes you will need to use (it is a helpful exercise to think through how we choose the input sizes):

Layer	**Input Size**	**Output Size**
Atomic Number Embedding	100	32
Aromaticity Embedding	2	4
Hybridization Embedding	8	4
Attached Hydrogens Embedding	8	8
Degree Embedding	8	8
Formal Charge Embedding	17	10
All Embed Layer (Linear)	(32 + 4 + 4 + 8 + 8 + 10)	n_embed

Table 1: Embedding Layer Sizes

Convolutions

For the message passing and node-update steps, there are many possible designs for a GNN's convolution operations to be performed on *node features* h_i and *edge features* e_{ij}. For this PSET, we will design a convolution that involves the features chosen in Part 1.2 z as initial node features h_i, and no edge features. Notice that even under this setting, we still need knowledge of node connectivities (i.e., the edges themselves) to pass messages between nodes. Each GNN layer operates on graph-structured data in two steps: a message step and an update step. The message step takes information from connected nodes, and the update function transforms these parameterized messages to update the features (embeddings) of each node:

$$\begin{aligned} m_i^t &= \sum_{j \in N(i)} m_{ij}^t = \sum_{j \in N(i)} \mathbf{MessageMLP^t}(h_i^{t-1} \cdot h_j^{t-1}) \\ h_i^t &= h_i^{t-1} + \mathbf{UpdateMLP^t}(m_i^t) \end{aligned} \tag{2}$$

Here, $\cdot$ is the element-wise multiplication and $j \in N(i)$ indicates the set of nodes that is connected to atom i. The superscript $t \in [0 \dots T]$ is the index of the layer; the subscript is index of the atom (node) in the graph. For each layer t, m_{ij}^t is the message from the edge set $(i, j) \in \mathcal{E}$ that affects the node embeddings in the next layer and h_i^t is the parametrized node embedding used to determine the message in the next layer. $\mathbf{MessageMLP^t} : \mathbb{R}^M \to \mathbb{R}^M$ and $\mathbf{UpdateMLP^t} : \mathbb{R}^M \to \mathbb{R}^M$ are MLPs that parameterize the messages and update atom (node) embeddings.

For this, you'll need a few additional tools that we've provided. In order to sum all the messages from $j \in N(i)$ onto each node i in Equation 2, use the `scatter_add()` function we've provided alongside clever PyTorch indexing into `h`. Do not use a for loop over either the atoms or the number of edges; that will be too slow for your GNN to run.

Final aggregation

After T convolutional layers, each node receives a parameterized embedding $h_i^T \in \mathbb{R}^M$. To map all the node embeddings h_i^T onto a predicted scalar value, you need to construct a **ReadoutMLP** : $\mathbb{R}^M \to \mathbb{R}$. The final property prediction layer has the form:

$$y = \sum_{i \in \{1,2,\dots|\mathcal{V}|\}} \mathbf{ReadoutMLP}(h_i^T) \tag{3}$$

Once you finish running the convolutions, you'll want to implement Equation 3. The challenge here is that the graph convolutional procedure operates on batched graph data, so you will need to split the final property prediction output $\mathbf{ReadoutMLP}(h_i^T)$ back into the original graphs to perform the summation; use `Natom` to determine the original size of each graph. To do this, use the `torch.split()` function to split the tensor based on the list of the number of atoms (indices) in each and then sum the embeddings of nodes within the same graph.

To aid you with these steps, we've provided examples of the use of both `scatter_add()` and `torch.split()` for you. You should not need any fancy PyTorch functions beyond the two that we've provided you. Remember to double-check the shape of your output from the GNN and make sure it's what you expect; you've already seen from previous problem sets that this can cause issues.

Overview of tasks

In the `GNN()` class, we have already implemented all the MLPs you need for each convolution stored in a `nn.ModuleList` which you can loop over for each convolution step to retrieve each model, as

well as the readout MLPs. Then, you will need to create the individual-feature embedding layers, as well as the first linear layer that provides an n_{embed} embedding per node. You will need to implement the `forward()` function, which does the actual computation given all these MLPs. To be explicit, in the forward call, you'll need to:

1. Embed each feature separately (and, translating the formal charge vector by +8 to ensure all values are non-zero)
2. Concatenate the features together
3. Create a $R^{n_{embed}}$ embedding
4. Perform all the convolutions
5. Use the readout layer to get a single feature value per node
6. Split the readout vector by the nodes of a graph (see above).
7. Sum all the readout values over each graph to yield a single scalar value per graph.

Implement the graph convolution model described above. Choose a hidden dimension of 64 ($M = 64$) and a depth of 3 ($T = 3$). You can specify M and T in the model as `n_embed` and `nconvs` in the `GNN()` class we defined for you.

Part 1.5.1 (grad only, 20 points)

Task: Complete the embedding steps, the convolution steps, and the final readout step + per-graph node aggregation.

Part 1.5.2 (undergrad only, 10 points)

Task: Complete the embedding steps and the final readout step + per-graph node aggregation. The convolution step has been taken care of for you using the GCN inheritance defined.

Part 1.6 (grad 5 points) Verify that your GNN preserves permutational invariance

When a GNN operates on a set of graphs, the nodes of a graph are arranged in a particular order. An important property that GNNs need to ensure is that the output should be the same given arbitrary node ordering. In other words, the final molecule-level output of GNNs should preserve *permutational invariance*.

Task: Verify that the GNN outputs respect permutation invariance by running the cell we have provided you. It operates on a small 4-node graph, generates a list of all possible permutation of node orders using `itertools.permutations`, and permutes the graph with the function `permute_graphs()` to get a re-ordered set of node and edge inputs. If you have implemented your GNN correctly, it should produce the same output for all the permuted graphs.

Part 1.7 (5 points) Train and test your GNN

Task 1: Train your GNN (undergrads can uncomment out the provided implementation) for 150 epochs with the train and validation loop we provided to you. The runtime of this training should not be more than 1-2 minutes on a T4 GPU. **Task 2**: Calculate the R^2 and render a scatter plot of predicted vs. true solubilities for both the training and test set.

Part 1.8 (5 points) Testing two "identical" molecules

Now that your GNN model can predict molecular solubilities, let's apply it to a pair of closely related molecules. The second molecule is a deprotonated version of the first, but otherwise have identical molecular connectivity. These subtle differences in their protonation states can significantly affect the intermolecular interactions with solvents like water. Solubility is sensitive to such changes, as variations in protonation influence how these molecules interact with the surrounding solvent environment.

Task 1: Inspect the two molecules we have provided. By visual inspection of these graphs, which of the atomic features that you encode in Part 1.2 would you expect to differ? What atomic features will remain the same? And, which molecule is recorded to be more soluble? (Bonus: can you explain why that molecule should be more soluble in terms of first-principles chemistry?)

Task 2: Process these SMILES into atom, edge, and atom count arrays the same way you did in 1.2, and pass these to your model to predict their solubility values. What happens? Is this expected? Explain why/why not.

Part 1.9 (grad 10 points) Obtain and compare node-level embeddings of these molecules

Using the above molecules, let's dig a little deeper into the predictions from the model.

Recall that we perform a final aggregation layer and summation over all nodes after all message-passing convolutions are done to obtain a final scalar molecular-level value. However, examining the final atom-level embeddings can reveal which atoms contribute most to the solubility scores.

To do so, we can design a "hook", which essentially lets you grab information that comes from any intermediate layer of the forward call before the final output, when a forward call is made. Since each intermediate layer in our GNN (after the initial concatenation of feature-specific embeddings) outputs an embedding per-atom, either of n_{embed} or 1, we can choose to look at the embeddings from any of these steps. For this problem, we'll keep things simple and look at the final per-atom embedding, which should be of width 1.

To create a hook, you'll need to create a "hook" function, which should be defined with three arguments: `module, input, output`. This function should store the `output` value in a global variable, which we have defined for you as `node_embeds`. This function will then be passed to `register_forward_hook` or `register_module_forward_hook`, like so:
`model.layer.register_forward_hook(hook_fn)`.

Task: Generate node-level embeddings for the two molecules using the hook functionality from the last layer, which should have an embedding width of 1 (i.e, each atom has one feature value associated with it, and your node embeddings should be dimension (11,1)). Analyze differences in the embeddings, making sure to match up comparisons based on the original atom indices. What differences do you notice? How do these individual values relate to the differences in the final solubility scores?

Part 2: Predicting IC_{50} with a 2D GNN

As a small extension of the above problem, we will investigate what happens when we try to learn a property that can be influenced by 3-dimensional information, such as conformation, with a 2D GNN. One such property is protein-ligand binding affinity, as these interactions depend not only on the specific atom types and connectivity but also on their geometric arrangement and proximity to interacting atoms.

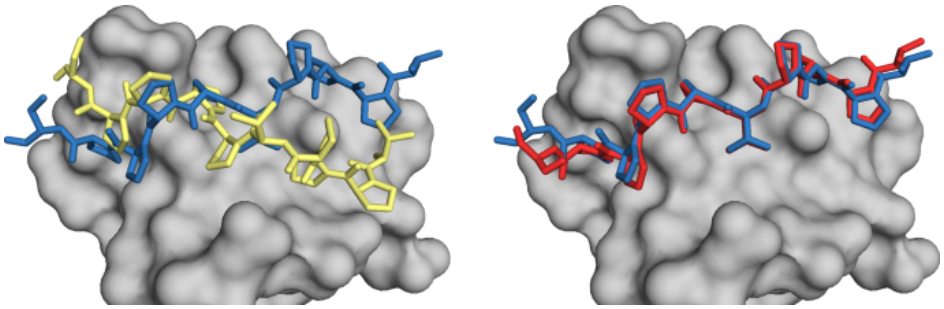

Figure 5: An example of two conformations of two ligands (left and right) in a protein pocket.

In experimental practice, binding affinity is instead measured by a phenotypic metric activity or function of that protein. One such measure is the IC_{50}, which is the concentration of that ligand required to inhibit 50% of a target protein's biological function. Since this value will of course vary by ligand/conformation and protein target, for this problem, we will work with ∼1000 ligands collected that were screened against one protein, human β-secretase 1(BACE-1)[3]. This simplification will allow us to abstract the contributions from the protein residues to this interaction and focus on the differences of ligands.

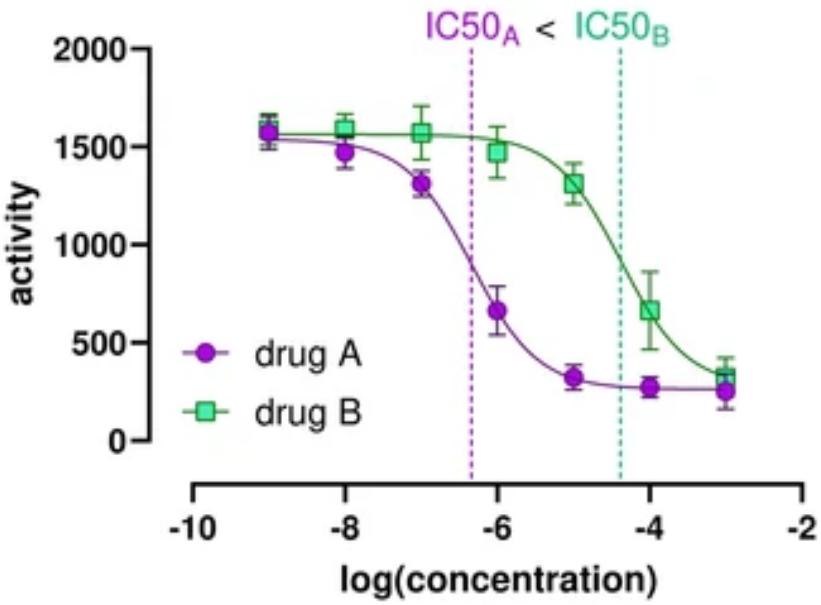

Figure 6: A example comparing two IC_{50} curves, each for a ligand against a target protein: with added concentration, the activity of the protein continues to drop, but at a steeper rate for drug A compared to drug B.

SMILES can also encode some 3D information: the presence of `@` and `@@` in SMILES strings are an indication of included stereoisomerism. For now, these will suffice for a basic inclusion of 3D information; as we'll discuss more carefully later in the class, conformers (various orientations of the same molecule) can also be expressed as a set of 3D coordinates.

Part 2.1 (10 points) Explore the data distribution in the dataset

As mentioned in the background, molecular conformation and thus stereochemistry can influence the binding affinity and thereby IC_{50} considerably. An unfavorable conformation can break important protein-ligand interactions. To get an idea of this property, let's view what kinds of stereochemistry are present in the data.

Task 1: Load the dataset and perform some exploratory data analysis.

1. What are the mean and variance of IC_{50} scores?
2. How many molecules with recorded stereochemistry do we have in the dataset?
   (a) To quickly identify recorded stereochemistry, we can "roundtrip" the molecule by making a `mol` from the original SMILES, then make two SMILES outputs from that `mol`: one that is an isomeric SMILES and a non-isomeric SMILES (i.e., one that retains any present chiral information), and compare the two output SMILES strings. If they are the same, the molecule didn't have any recorded stereochemistry. Pass `isomericSmiles=True` and `isomericSmiles=False` to the `Chem.MolToSmiles()` function to get the two SMILES strings for this.
3. How many molecules could have nontrivial stereochemistry recorded, but don't actually have any information recorded?
   (a) Hint: you can find the number of chiral centers, i.e. atomic centers where nontrivial stereochemistry can be labeled, with `Chem.FindMolChiralCenters(mol, includeUnassigned=True)`. Then compute how many molecules have chiral centers but do not have stereochemistry tracked.

Part 2.2 (5 points) Process this data into feature lists

Task 1: Load this dataset with pandas, and process the SMILES with your code from Part 1.2 to encode them into 2D graphs. Use the 'pIC50' column for the y values you will predict in this task. Split the data according to the "Split" column in the file rather than generating your own dataset splits. Finally, set up the `GraphDataset` objects and `GraphDataLoaders` as you did in Part 1.4. Use a batch size of 32.

Part 2.3 (10 points) Train and evaluate your GNN's performance overall and by chiral center count

Task 1: Using your same GNN implementation from Problem 1, train a new GNN to predict the IC_{50} value for these molecules for 100 epochs. The runtime of this training should not be more than 1-2 minutes on a T4 GPU. Like in Problem 1, you should be training only on the train set, and using the validation set for tracking generalization.

Task 2: Upon completion of training, for each of the train, validation, and test sets, report the RMSE and plot a scatterplot of predictions.

Task 3: Stratify performance of the samples in your test model by number of chiral centers. Seaborn is another plotting library that, when given a dataframe, can produce visualizations handily when given column references. Make a `sns.boxplot` that shows the distribution of absolute prediction errors for the test set, stratified by the number of chiral centers present in each molecule. This plot allows you to visually compare how the spread of prediction errors varies with the count of chiral centers in the molecules. Are there any trends?

Part 2.4 (10 points) How do two stereoisomers compare?

The dataset includes multiple sets of stereoisomers (molecules that have the same 2D connectivity but differ in their stereochemistry). You may have potentially stumbled upon these when exploring the data in 2.1.

Task 1: Amongst the subset of molecules that have recorded stereochemistry, find groups (of size more than 1) of molecules that share the same 2D connectivity. They do not need to be limited to a specific split of the dataset. How many such groups are there? Save this count to `num_stereogroups`, and the entries corresponding to these groups in `df_stereogroups` for use in the downstream questions. You may find `{df}.groupby(column)` and/or `{df}.value_counts(column)` helpful.

Hint: We can reformat the SMILES to be canonicalized and without any stereochemistry by using `Chem.MolToSmiles(mol, canonical=True, isomericSmiles=False)`.

Task 2: For all of the entries that are found in these groups, process each of the original SMILES (not the `clean_smiles` strings) into atom, edge, and atom count arrays the same way you did in 2.2, and pass these to your model to predict their IC_{50} values. Feel free to do this one by one in a for loop, to make sure that you record the predicted pIC50s correctly for each molecule (we don't want you to get hung up on manual testing of your model!). Store each of these predicted pIC50 values back into your `df_stereogroups` dataframe.

Important note: If your molecules happen to be tautomers of each other (i.e., have a migration of hydrogen atoms around the molecule), for the purposes of this problem, standardize them to be the same tautomer to remove protonation state as a confounding variable like so: `corr_mol = Chem.MolFromInchi(Chem.MolToInchi(orig_mol))`. Make sure that you do not inadvertently remove recorded stereochemistry.

Task 3: Compare your pIC50 values to the recorded pIC50 values. How do the predicted values compare to the true IC50 values? Pick one of the groups where you can demonstrate that the GNN has not properly learned chirality (i.e., predicts the same value when it shouldn't).

Important note: To simplify assumptions for this problem, select two molecules from the same group where both molecules have recorded, differing stereochemistry. Although a molecule without recorded stereochemistry may be present in the group, the most meaningful comparison will occur between two molecules that explicitly have documented stereochemical differences.

References

[1] Sorkun, M. C., Khetan, A. & Er, S. AqSolDB, a curated reference set of aqueous solubility and 2D descriptors for a diverse set of compounds. *Scientific Data* (2019).

[2] Galvez, J., Garcia-Domenech, R. & Castro, E. Molecular topology in QSAR and drug design studies. QSPR-QSAR Studies on Desired Properties for Drug Design. *Research Signpost* 63–94 (2010).

[3] Wu, Z. *et al.* MoleculeNet: A benchmark for molecular machine learning. *arXiv [cs.LG]* (2017).